Yemekhane Otomasyonu – Dönem Ödevi

Web Teknolojileri · Oct 20, 2026 · @Burak Yılmaz

Amaç

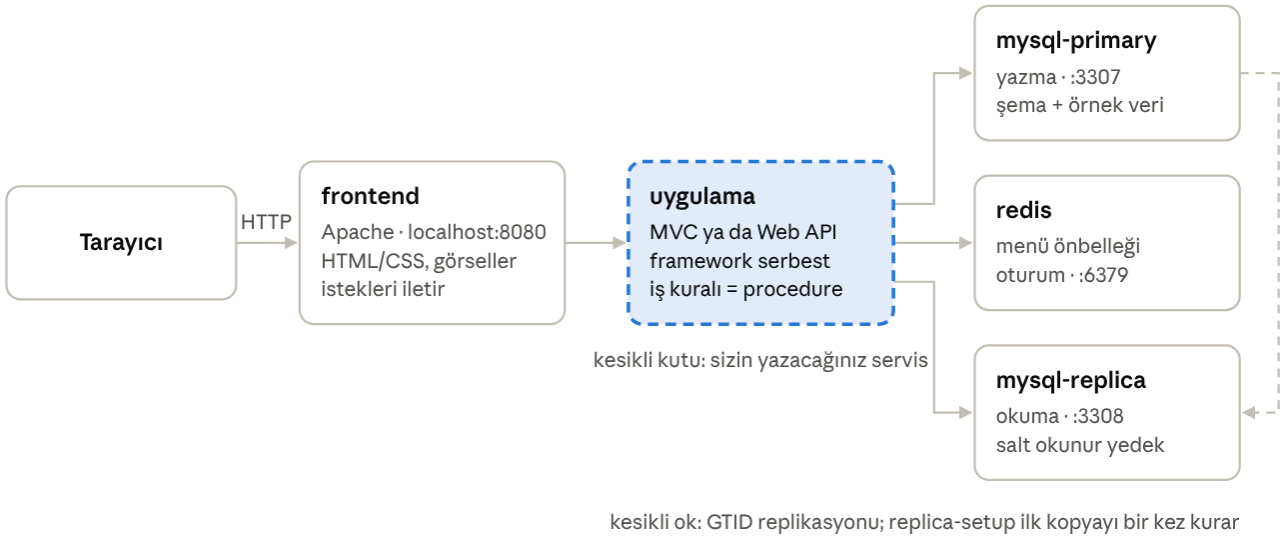
Her öğrenci bireysel olarak, kampüs yemekhanesi için menü, rezervasyon ve bakiye işlemlerini yöneten bir web uygulamasını tamamen Docker container'ları üzerinde kuracak. Ödev 20 Ekim 2026'da verilir, süre üç haftadır. Ödev dört aşamadan oluşur; son aşamada yemekhane müdürlüğünden gelen bir değişiklik talebi hem veritabanına hem arayüze uygulanır.

Ödev sonunda öğrenci:

- Birden fazla servisi docker compose ile tek komutla ayağa kaldırır ve servisler arası ağ, volume, healthcheck kavramlarını uygular.
- HTML5 ve CSS3 ile duyarlı (responsive) bir arayüz tasarlar, MVC ya da Web API mimarisinden birini seçerek katmanlı bir uygulamaya bağlar.
- Verilen ilişkisel MySQL şeması üzerinde sorgu, stored procedure ve view kullanır.
- MySQL kaynak-replika (source-replica) replikasyonunu kurar, okuma ve yazma trafiğini ayırır.
- Redis'i önbellek ve oturum deposu olarak kullanır, önbellek geçersizleştirmeyi doğru zamanda yapar.
- Çalışan bir sistemde şema değişikliğini migration ile uygular ve değişikliği arayüze yansıtır.

Sistem mimarisi

Sistem altı container'dan oluşur; beşi başlangıç paketinde hazırdır, Uygulamayı (MVC ya da Web API) siz yazarsınız. Tarayıcı veritabanına doğrudan bağlanamaz; veriye yalnızca uygulamanın veri katmanı ulaşır.



sistem mimarisi · 6 container

Yazma işlemleri kaynak sunucuya, okuma işlemleri replikaya gider; replika aynı zamanda kaynak çöktüğünde verinin yedeğidir.

Verilen dosyalar ve veritabanı

Başlangıç paketi `docker compose up -d` ile çalışır durumda gelir: veritabanı şeması, örnek veri, replika ve Redis hazırdır. Sizin yazacağınız kısım uygulama ve arayüzdür.

Dosya	İçerik
docker-compose.yml	frontend, mysql-primary, mysql-replica, replica-setup, redis servisleri; uygulama servisi yorum satırında
db/init/01_sema.sql	12 tablo, 3 view, 4 stored procedure
db/init/02_ornek_veri.sql	30 kullanıcı, 52 yemek, 84 menü, yaklaşık 365 rezervasyon; tarihler kurulum gününe göre üretilir
db/replica-setup.sh	replikayı bir kez kurar ve salt okunur yapar
frontend/, apache/	Apache 2.4'ün yayınladığı HTML/CSS ve uygulamaya yönlendirme (MVC ve Web API için hazır ayar)
README.md	bağlantı bilgileri, doğrulama komutları, sık sorunlar

Şemadaki tablolar:

Tablo	Ne tutar	Dikkat edilecek kural
kullanicilar	öğrenci, personel, yönetici	bakiye negatif olamaz; öğrenci numarası yalnızca öğrencide bulunur
yemekhaneler	Merkez (400 kişi), Mühendislik (60 kişi)	kapasite rezervasyonu sınırlar
ogunler	Kahvaltı, Öğle, Akşam	her öğünün rezervasyon son saati var (07:00, 10:00, 15:00)
yemek_kategorileri, yemekler	7 kategori, 52 yemek, kalori	menüde kategori sırasıyla gösterilir
alerjenler, yemek_alerjenleri	14 alerjen, yemek-alerjen eşleşmesi	çoka çok ilişki
menuler, menu_yemekleri	yemekhane + tarih + öğün başına tek menü	öğrenci ve personel fiyatı ayrı
rezervasyonlar	aktif, kullanıldı, iptal	kişi başına menü başına tek kayıt
bakiye_hareketleri	yükleme, rezervasyon, iade	bakiye her zaman hareketlerin toplamına eşittir
degerlendirmeler	menüye 1–5 puan ve yorum	kişi başına menü başına tek oy

Para hareketi yapan işlemler veri katmanında SQL ile yeniden yazılmaz, stored procedure çağrılır: `sp_rezervasyon_yap`, `sp_rezervasyon_iptal`, `sp_bakiye_yukle`, `sp_turnike_gecis`. Hata durumunda yordamlar Türkçe mesajla `SQLSTATE 45000` döndürür (ör. “Bakiye yetersiz.”); uygulama bu mesajı arayüze iletmelidir. Hazır view'lar: `v_menu_detay`, `v_menu_doluluk`, `v_menu_puan`.

Aşamalar

Son teslim: Nov 10, 2026, saat 23.59. Dört aşama tek seferde teslim edilir; aşamalar çalışma sırasıdır ve her biri bir öncekinin üzerine kurulur. Ara sınav haftası (31 Ekim – 8 Kasım) bu sürenin içindedir; planınızı buna göre yapın.

Aşama 1 · Altyapı

Başlangıç paketini çalıştırın ve her servisin beklendiği gibi çalıştığını kanıtlayın.

- [ ] `docker compose ps` çıktısında tüm servisler healthy, replica-setup başarıyla çıkmış.
- [ ] Kaynakta yapılan bir bakiye yüklemesi replikada görünüyor.

- [ ] Replikaya yazma denemesi read-only hatasıyla reddediliyor.
- [ ] Replika container'ı durdurulup yeniden başlatıldığında aradaki değişiklikleri yakalıyor.
- [ ] Raporda her maddenin komutu ve çıktısı var.

Aşama 2 · Uygulama

İki yoldan birini seçin ve raporun ilk sayfasında belirtin. Framework her iki yolda da serbesttir (ASP.NET Core, Spring, Laravel, Django, Express gibi).

- **MVC:** Sayfaları uygulamanın View katmanı HTML5 + CSS3 ile üretir. CSS ve görseller Apache'den sunulur, diğer istekler uygulamaya iletilir.
- **Web API + arayüz:** API yalnızca JSON döner. HTML5 + CSS3 arayüz frontend container'ında durur ve veriyi JavaScript `fetch` ile alır; React gibi bir çerçeve kullanılmaz. Apache `/api/` isteklerini API'ye iletir.

Hangi yolu seçerseniz seçin uygulamayı compose dosyasına `uygulama` servisi olarak ekleyin. Veri erişimi ayrı bir katmanda durur; Controller içinde SQL, arayüz kodunda iş kuralı bulunmaz. Raporda bir isteğin uygulamanız içindeki yolunu (route → Controller → veri katmanı → yanıt) kendi kodunuzdan bir örnekle gösterin.

Arayüzde bulunması gereken ekranlar:

1. Giriş ve çıkış (e-posta + şifre, bcrypt doğrulaması).
2. Günün menüsü: yemekhane ve öğün seçimi, yemekler, toplam kalori, vejetaryen ve alerjen bilgisi.
3. Haftalık menü ve rezervasyon: boş yer sayısı, kullanıcının rolüne göre fiyat, son saat geçtiyse pasif düğme.
4. Rezervasyonlarım: aktif, kullanıldı, iptal durumları; süresi dolmamışsa iptal.
5. Bakiye: güncel bakiye, hareket geçmişi, bakiye yükleme.
6. Değerlendirme: kullanılmış bir menüye 1–5 puan ve yorum.
7. Yönetici paneli: menü oluşturma ve düzenleme, menü bazında doluluk ve ortalama puan.

Kurallar: yazma işlemleri kaynak sunucuya, listeleme ve raporlar replikaya gider. Rezervasyondan hemen sonra gösterilen bakiye kaynaktan okunur; raporda bunun nedenini (replikasyon gecikmesi) bir paragrafla açıklayın. Arayüz telefon genişliğinde (yaklaşık 400 px) kullanılabilir olmalıdır.

Aşama 3 · Redis

Redis iki iş için kullanılır ve ikisi de rapordaki `redis-cli` çıktılarıyla gösterilir.

Görev	Anahtar	Süre	Ne zaman silinir
Menü önbelleği	`menu:{yemekhane_id}:{tarih}:{ogun_id}`	o günün sonuna kadar	yönetici o menüyü değiştirdiğinde
Oturum	`oturum:{token}`	30 dakika, her istekte yenilenir	çıkış yapıldığında

- [ ] İlk istek veritabanından, ikincisi Redis'ten geliyor (uygulama günlüğünde görünür).
- [ ] Menü güncellenince eski kayıt siliniyor, arayüz yeni menüyü gösteriyor.
- [ ] Uygulama container'ı yeniden başlasa da kullanıcının oturumu düşmüyor.

Aşama 4 · Değişiklik talebi

Yemekhane Müdürlüğü sistem çalışırken iki yeni özellik istiyor:

- **Yarım porsiyon:** öğle ve akşam menülerinde yarım porsiyon seçilebilir, ücret tam fiyatın %60'ıdır. Kahvaltıda yarım porsiyon yoktur. Geçmiş rezervasyonlar tam porsiyon sayılır.
- **Alerjen uyarısı:** kullanıcı profilinde alerjenlerini seçer; menüde bu alerjenleri içeren yemekler uyarıyla gösterilir.

Teslim edilecekler:

- [ ] Tek bir migration dosyası (`04_degisiklik.sql`): yeni tablo ve sütunlar, güncellenmiş `sp_rezervasyon_yap`. Mevcut veriyi bozmadan kaynakta çalışır.
- [ ] Değişikliğin replikaya da geçtiğinin kanıtı (`SHOW CREATE TABLE` çıktısı replikadan).
- [ ] Uygulama ve arayüz: rezervasyonda porsiyon seçimi, profil sayfasında alerjen seçimi, menüde uyarı.
- [ ] Redis'te eski yapıdaki menü kayıtlarının temizlenmesi.

Teslim ve değerlendirme

Ödev 100 puan üzerinden değerlendirilir; tüm aşamalar son teslimde birlikte puanlanır.

Teslim kuralları:

- Ödev bireyseldir. Her öğrenci kendi GitHub deposunda çalışır; commit geçmişi üç haftaya yayılmış olmalıdır.
- Teslim, depo bağlantısının sisteme yüklenmesiyle yapılır. Her aşamanın bittiği commit bir etiketle işaretlenir: `asama-1`, `asama-2`, `asama-3`, `asama-4`.
- Depo kökünde `docker compose up -d` komutu ek adım olmadan tüm sistemi çalıştırmalıdır. Çalışmayan kısım puan alamaz.

- `rapor/` klasörüne, aşama aşama komutları, çıktıları ve ekran görüntülerini içeren kısa bir PDF eklenir.
- Teslimden sonra her öğrenci 10 dakikalık canlı gösterim yapar ve kodunun herhangi bir kısmını açıklayabilmelidir.

Kalem	Puan
Aşama 1: compose, replikasyon doğrulaması, rapor	15
Aşama 2: katman ayrımı (MVC ya da API) ve stored procedure kullanımı	15
Aşama 2: HTML/CSS arayüz, yedi ekran, telefon uyumu	15
Aşama 2: okuma/yazma ayrımı ve hata mesajlarının arayüze taşınması	10
Aşama 3: Redis önbellek ve oturum, doğru geçersizleştirme	15
Aşama 4: migration, replikaya geçiş, uygulama ve arayüz güncellemesi	20
Kod kalitesi, commit geçmişi, canlı gösterim	10
Toplam	**100**

İki öğrencinin kodu büyük ölçüde aynıysa ikisi de ilgili aşamadan sıfır alır. Geç teslim kuralı: [ders sayfasında belirtilecek].